Mathematical Background and Implementation Guide

N2SCFTDB / n2_theory_index.py

FORM and LiE computation of four-dimensional $\mathcal{N} = 2$ indices
Simple and semisimple product gauge groups

Property/database revision: 12 September 2026

Purpose. The canonical implementation computes the flavor-unrefined right superconformal index of a Lagrangian four-dimensional $\mathcal{N} = 2$ gauge theory. FORM performs exact truncated series algebra. LiE decomposes individual Adams operations and intermediate products. Python pairs dual irreps to extract the final singlet, independently for every simple gauge factor. SQLite stores exact parsed FORM expansions, decompositions and final singlet coefficients. Sage supplies input/representation data, the dual-label convention and the exact returned polynomial ring.

The index/cache chapters describe commit `04bbe21` and its dated measurements. This revision adds separate property/index calculations, the schema-7 database worker and independent precision upgrades. The mathematical equations are retained; the companion reference gives package-wide context.

Contents

1 Scope and program architecture

The supported gauge algebra is a finite semisimple algebra

$$\mathfrak{g} = \bigoplus_{a=1}^{s} \mathfrak{g}_a, \qquad G = \prod_{a=1}^{s} G_a, \tag{1}$$

where every $\mathfrak{g}_a$ has Cartan type $A_r, B_r, C_r, D_r, E_6, E_7, E_8, F_4$, or G_2. The compact group convention is the simply connected one used by the shared Lie-algebra module; in particular C_r is written $\mathrm{Sp}(r)$. Abelian gauge factors are outside the current input model.

The calculation has four layers:

1. `check_input_data` validates the existing simple- or product-group JSON convention and resolves names to Dynkin labels.
2. `_build_form_program` constructs the truncated plethystic exponential with formal character symbols.
3. `CharacterDecompositionCache` asks LiE for the necessary virtual character decompositions and caches decompositions and singlet coefficients in SQLite.
4. `_project_terms` extracts the gauge singlet factor by factor and `_to_sage_polynomial` packages the exact answer.

This division is important: FORM never needs to know a root system, while LiE never needs to manipulate the fugacities t, y, u.

2 Index convention and single-letter functions

Choose the supercharge defining the right index. With the conventions of Gadde, Pomoni, and Rastelli, the BPS condition and trace are [1]

$$\delta_R = \Delta - 2\bar{j} - 2R + r = 0, \tag{2}$$

$$\mathcal{I}(t, y, v) = \mathrm{Tr}_{\delta_R=0}(-1)^F t^{2(\Delta+\bar{j})} y^{2j} v^{-r-R}. \tag{3}$$

The code substitutes

$$v = u^2, \tag{4}$$

so every computational power of u is integral.

The vector and half-hypermultiplet single-letter indices are [1]

$$f_V(t, y, u) = \frac{t^2u^2 - t^4u^{-2} - t^3(y + y^{-1}) + 2t^6}{(1 - t^3y)(1 - t^3y^{-1})}, \tag{5}$$

$$f_H(t, y, u) = \frac{t^2u^{-1} - t^4u}{(1 - t^3y)(1 - t^3y^{-1})}. \tag{6}$$

The denominator counts the two compatible derivatives:

$$\frac{1}{(1 - t^3y)(1 - t^3y^{-1})} = \sum_{p,q\geq 0} t^{3(p+q)} y^{p-q}. \tag{7}$$

FORM uses a finite rectangular sum with $0 \leq p, q \leq \lfloor D/3 \rfloor$. Terms whose combined t degree exceeds D are removed automatically.

3 Matter characters for simple and product groups

For a product group, an irreducible external tensor product is

$$\mathbf{R}_h = \boxtimes_{a=1}^{s} R_{h,a}, \qquad \chi_{\mathbf{R}_h}(g_1, \ldots, g_s) = \prod_{a=1}^{s} \chi_{R_{h,a}}(g_a). \tag{8}$$

An omitted representation in the JSON input is the singlet, whose character is one.

The adjoint representation of the product algebra is a direct sum,

$$\mathrm{ad}_{\mathfrak{g}} = \bigoplus_{a=1}^{s} (\mathbf{1}, \ldots, \mathrm{ad}_{\mathfrak{g}_a}, \ldots, \mathbf{1}), \qquad \chi_{\mathrm{ad}_{\mathfrak{g}}} = \sum_{a=1}^{s} \chi_{\mathrm{ad}_a}. \tag{9}$$

This explains why vector-multiplet characters are added, whereas a bifundamental hypermultiplet character is multiplied across its gauge factors.

For multiplicity N_h, the character measured in half-hyper units is

$$\widehat{\chi}_h = \begin{cases} N_h \chi_{\mathbf{R}_h}, & \text{half hyper,} \\ N_h (\chi_{\mathbf{R}_h} + \chi_{\overline{\mathbf{R}}_h}), & \text{full hyper.} \end{cases} \tag{10}$$

Conjugation is simultaneous and factorwise:

$$\overline{\mathbf{R}}_h = \boxtimes_{a=1}^{s} \overline{R}_{h,a}. \tag{11}$$

Thus a full hyper in $R_1 \boxtimes R_2$ contains $R_1 \boxtimes R_2$ and $\overline{R}_1 \boxtimes \overline{R}_2$; it does not contain the two mixed tensor products unless they are separate matter entries.

4 Plethystic exponential and exact cutoffs

With flavor fugacities unrefined, define the representation-valued letter index

$$\mathcal{L}(t, y, u; g) = f_V(t, y, u) \sum_{a=1}^{s} \chi_{\mathrm{ad}_a}(g_a) + f_H(t, y, u) \sum_{h} \widehat{\chi}_h(g). \tag{12}$$

The free-field index before gauge projection is the plethystic exponential [1, 2]

$$\mathcal{Z}(t, y, u; g) = \mathrm{PE}[\mathcal{L}] = \exp E(t, y, u; g), \tag{13}$$

$$E(t, y, u; g) = \sum_{j=1}^{\infty} \frac{1}{j} \mathcal{L}(t^j, y^j, u^j; g^j). \tag{14}$$

Since every letter begins at t^2, a result through t^D needs only

$$1 \le j \le M = \left\lfloor \frac{D}{2} \right\rfloor. \tag{15}$$

The same lower bound implies that only $E^k/k!$ with $k \le M$ can contribute. Consequently both the Adams sum and the ordinary exponential have finite, exact cutoffs at the requested order.

5 How FORM represents and expands the index

5.1 Formal character symbols

Each distinct pair "gauge-factor position, Dynkin labels" receives a commuting FORM function $C_p(j)$. Mathematically it denotes the j-th Adams operation

$$C_p(j) = \psi^j(\chi_{R_p}), \qquad (\psi^j \chi)(g) = \chi(g^j). \tag{16}$$

For a bifundamental, FORM therefore sees $C_p(j)C_q(j)$. It expands only the coefficients and products of these formal objects; their Lie-theoretic decomposition is postponed until after the series is complete.

At Adams degree j, `_build_form_program` constructs

$$\mathcal{L}_j = f_V(t^j, y^j, u^j) \sum_{p \in \mathcal{V}} C_p(j) + f_H(t^j, y^j, u^j) \sum_m N_m \prod_{p \in m} C_p(j), \tag{17}$$

where $\mathcal{V}$ contains the adjoint character of every gauge factor and the matter monomials already include the full-hyper conjugate term from Eq. (10).

5.2 The generated FORM program in plain language

The central generated block has the following form:

Listing 1: Schematic FORM program generated by Python.

```
S m,n,y,idx1,idx2,j,z,u,t(:D);
CF d,C0,C1,...;
PolyRatFun d;
Function Kvec,Khyp;

L J=sum_(idx1,0,D/3,m^idx1)
   *sum_(idx2,0,D/3,n^idx2);
id m=t^3*y;
id n=t^3/y;
.sort

L itotal=sum_(j,1,D/2,(vector terms + hyper terms)/j);
id Kvec(t?,y?,u?)=J*(t^2*u^2-t^4/u^2-t^3*y-t^3/y+2*t^6);
id Khyp(t?,y?,u?)=J*(t^2/u-t^4*u);
.sort
```

FORM construct	Meaning in this calculation
`S ... t(:D);`	Declares commuting symbols. The maximum-power declaration automatically discards $t^{D+1}, t^{D+2}, \ldots$ during term normalization [4, Sec. 7.143].
`CF d,C0,...;`	Declares commuting functions. The $C_p(j)$ objects are formal Adams-operated characters.
`PolyRatFun d;`	Makes `d(numerator,denominator)` the exact rational coefficient container, so equal monomials combine without floating point arithmetic [4, Sec. 7.114].
`L name=...;`	Creates a local FORM expression. `J`, `itotal`, `I`, and `result` are successive exact expressions.
`sum_(j,a,b,X)`	Generates the finite sum $\sum_{j=a}^{b} X$; the last argument is the summand [4, Sec. 8.67].
`id lhs=rhs;`	Applies an algebraic substitution. A question mark denotes a wildcard, so `Kvec(t?,y?,u?)` matches arguments such as `Kvec(t^j,y^j,u^j)` [4, Secs. 7.70–7.71].
`.sort`	Ends a FORM module, expands products, combines identical terms, and enforces the t cutoff before the next module [4, Sec. 2.3].

5.3 Why the auxiliary symbol z gives the exponential

Let $E =$ `itotal` and $M = \lfloor D/2 \rfloor$. The code begins with $I = zE$. For $k = 2, \ldots, M$ it applies

$$z \longmapsto 1 + \frac{zE}{k}. \tag{18}$$

After the k-th step, induction gives

$$I_k = \sum_{r=1}^{k-1} \frac{E^r}{r!} + z\frac{E^k}{k!}. \tag{19}$$

The final substitution $z \mapsto 1$, followed by adding the vacuum term, yields

$$1 + I_M = \sum_{r=0}^{M} \frac{E^r}{r!} = \exp(E) + O(t^{D+1}). \tag{20}$$

This is an ordinary exponential of a representation-valued series; Bose and Fermi statistics have already entered through the signed single-letter functions.

6 Adams operations and LiE decomposition

6.1 The representation-ring calculation

For each irrep R and exponent vector $\mathbf{n} = (n_1, n_2, \ldots)$, the cache stores the virtual decomposition

$$D_R(\mathbf{n}) = \mathrm{Decomp}\left[\bigotimes_{j\geq 1} (\psi^j R)^{\otimes n_j}\right]. \tag{21}$$

An Adams operation scales every weight by j and can produce a virtual representation with negative coefficients [3, Secs. 3.4, 4.5]. For the A_1 fundamental,

$$\psi^2(\mathbf{2}) = \mathbf{3} - \mathbf{1}, \tag{22}$$

which LiE prints as a signed decomposition polynomial.

6.2 The LiE commands in plain language

LiE expression	Meaning
`Adams(2,[1],A1)`	Decomposes ψ^2 of the A_1 irrep with label [1]; the result is `-1X[0]+1X[2]` [3, Sec. 4.5].
`tensor([1],[1],A1)`	Returns `1X[0]+1X[2]` for $V_{[1]} \otimes V_{[1]}$ [3, Sec. 4.5].
`tensor(p,q,A1)`	Tensors two decomposition polynomials. The revised projector uses this for intermediate products only.
`tensor(p,q,[0],A1)`	LiE's targeted trivial coefficient operation. The revised projector instead performs the final dual-irrep pairing in Python.
`maxnodesN/maxobjectsN`	Raises LiE node/object limits. The code retries with these settings only after a failed ordinary run, avoiding eager large-allocation startup cost [3, Sec. 2.3].

A decomposition polynomial has the form

$$p = \sum_{\lambda \in P_+} m_\lambda X[\lambda_1, \ldots, \lambda_r], \qquad m_\lambda \in \mathbb{Z}. \tag{23}$$

The parser preserves the signed integer m_λ. It also removes LiE's optional "New tree space with maximum number of nodes" notification by recognizing the message prefix, rather than by deleting a fixed number of output characters.

7 Gauge-singlet projection for product groups

The gauge index is the Haar projection

$$\mathcal{I}(t,y,u) = \int_G d\mu_G(g)\, Z(t,y,u;g). \tag{24}$$

Equivalently, it is the multiplicity of the trivial representation in every coefficient. After FORM expansion, a typical formal-character monomial is

$$\mathcal{C} = \prod_p \prod_{j\geq 1} C_p(j)^{n_{p,j}}. \tag{25}$$

If $a(p)$ is the gauge factor associated with character p, define the virtual G_a representation

$$X_a(\mathcal{C}) = \bigotimes_{p:\, a(p)=a} \bigotimes_{j\geq 1} (\psi^j R_p)^{\otimes n_{p,j}}. \tag{26}$$

Because both the group and normalized Haar measure factorize,

$$d\mu_G = \prod_{a=1}^{s} d\mu_{G_a}, \tag{27}$$

$$\mathrm{mult}_{\mathbf{1}_G}(\mathcal{C}) = \prod_{a=1}^{s} \mathrm{mult}_{\mathbf{1}_a}(X_a(\mathcal{C})). \tag{28}$$

This is the precise rule implemented by `_project_terms`. It partitions a FORM term by gauge-factor position, batches distinct character-product singlet requests for each factor, and multiplies the returned integers. Empty character products have singlet multiplicity one. SQLite is checked before decomposition.

7.1 Exact pairing from character orthogonality

For compact G and normalized Haar measure,

$$\int_G \chi_\lambda(g)\overline{\chi_\mu(g)}\, d\mu(g) = \delta_{\lambda\mu}, \qquad \chi_{\mu^*} = \overline{\chi_\mu}. \tag{28a}$$

Thus for two partial virtual characters,

$$A = \sum_\lambda a_\lambda \chi_\lambda, \quad B = \sum_\mu b_\mu \chi_\mu, \qquad \boxed{[\mathbf{1}](AB) = \sum_\lambda a_\lambda b_{\lambda^*}.} \tag{28b}$$

Character orthogonality is Corollary 35.9 of [6]. Equation (28b) follows by substitution and linearity, including negative coefficients. Equivalently, $(V\otimes W)^G \simeq \mathrm{Hom}_G(V^*, W)$; the latter is the space of linear maps commuting with the group action. Schur's lemma selects dual irreps. The singlet functional here is bilinear; an ordinary coefficient dot product would incorrectly pair complex irreps with themselves.

7.2 Split before decomposition

A symbolic input `(labels,(n1,n2,...))` stands for $\prod_j \psi^j(R)^{n_j}$. The revised procedure is:

1. Remove trivial-character factors, since $\psi^j(\mathbf{1}) = \mathbf{1}$.
2. Expand exponents into individual Adams factors and split them into two approximately balanced parts A and B.
3. Fully decompose the needed *partial* products, reusing persistent and in-memory entries. Individual Adams operations remain indivisible.
4. Apply (28b) using sparse dictionary lookups and save the resulting integer.

The splitter's cost estimate for an atom is

$$c(\lambda, j) = j \max\left(1, \sum_i \lambda_i\right). \tag{28c}$$

It places larger estimates first on the currently lighter side, preferring an already-present base irrep when the two costs tie. This source-derived heuristic balances approximate highest-weight sizes, not exact tensor complexity, and carries no optimality guarantee. Repeated factors of one base irrep can be split across the two sides before their complete product is decomposed.

For example, $\psi^4(\text{adj})\psi^5(\text{adj})$ needs only the two individual Adams expansions and their pairing. In contrast, $\psi^3(\text{adj})^3$ leaves the intermediate $\psi^3(\text{adj})^2$, which can remain expensive. The full decomposition API is still available when explicitly requested; the singlet path avoids tensor-decomposing the complete product.

7.3 Dual labels and signed examples

The highest weight of the dual irrep is

$$\lambda^* = -w_0\lambda. \tag{28d}$$

The map permutes fundamental weights [6, Prop. 32.1]. `_dual_permutation` obtains that permutation with the existing Sage duality helper and caches it; `_singlet_pair` applies it to label tuples. See also Sage's `dual()` and `inner_product()` [7]. The pairing iterates over the smaller support, using $O(r\min(N_A, N_B))$ label operations. The costly step can still be constructing A or B.

For $SU(3)$, $(a,b)^* = (b,a)$, so $[\mathbf{1}](\mathbf{3}\otimes\overline{\mathbf{3}}) = 1$ but $[\mathbf{1}](\mathbf{3}\otimes\mathbf{3}) = 0$. Actual conjugate orientations must remain in cache keys, even where theory enumeration identifies conjugates. For $SU(2)$, $\mathbf{2}\otimes\mathbf{2} = \mathbf{1}\oplus\mathbf{3}$ implies $[\mathbf{1}](\mathbf{2}^{\otimes 4}) = 1^2 + 1^2 = 2$. Pseudoreality adds no sign to this pairing. Signed Adams coefficients are different: from (22), $[\mathbf{1}](\psi^2\mathbf{2}) = -1$, while $[\mathbf{1}]((\psi^2\mathbf{2})^2) = (-1)^2 + 1^2 = 2$.

8 SQLite caches and complete Adams-cache builder

For the exponent vector in Eq. (21), define its total Adams order

$$q(\mathbf{n}) = \sum_{j \geq 1} j n_j. \tag{29}$$

The default local file is `<project-root>/char_decomposition_cache.db`. Python's `sqlite3` stores both full character decompositions and final singlet integers. The schema has `user_version=1`, independently of the MySQL theory database. There is no combined product-group decomposition table; identical simple factors reuse the same entries.

Table	Columns and primary key
`character_decompositions`	`algebra TEXT`, `dynkin_labels TEXT`, `adams_powers TEXT`, `adams_order INTEGER`, `terms_json TEXT`. Key: `(algebra,dynkin_labels,adams_powers)`.
`singlet_coefficients`	`algebra TEXT`, `product_key TEXT`, `coefficient TEXT`. Key: `(algebra,product_key)`.

All columns are non-null, and both tables use `WITHOUT ROWID`. Label and power tuples are compact JSON text; signed coefficients are decimal strings, avoiding SQLite's 64-bit integer limit [9]. Decomposition payloads are lists of `[labels,"coefficient"]` pairs. The full power tuple is required: total Adams order alone does not distinguish, for example, $R^{\otimes 2}$ from $\psi^2(R)$.

Symbolic singlet keys contain `["characters",canonical_product]`. Repeated base labels merge by adding exponents; factors sort in descending order. Already-decomposed inputs use a separate `"decompositions"` namespace. Zeros and negative coefficients are valid cache hits. Returned decompositions are copies, so caller mutation does not change a cached entry.

8.1 Lookup order and concurrency

A singlet lookup checks thread-local memory, then SQLite, before requesting any partial decomposition. Same-base-irrep partial products use persistent entries; mixed partial decompositions live only in the thread-local `partial_products` dictionary. Only the final complete-product scalar is persisted as a singlet.

Each thread owns a connection. PID checks avoid inherited connections; before starting a process pool the parent closes its connection. Workers calculate independent requests, and the parent batches completed writes by dependency level. Sequential batches flush after 64 entries and at completion. LiE work occurs outside write transactions. Failed calculations do not cache a guessed zero.

WAL allows concurrent readers and a writer, while writers remain serialized; it assumes a local filesystem [8]. The code uses a 30-second busy timeout and bounded retries. Python connection contexts manage transactions; this cache's context manager additionally closes the connection and clears thread-local entries [10].

Legacy JSON file discovery/import and `cache_path()` are removed. The new projection preserves existing SQLite keys and schema; no migration is needed. FORM expansion now has its own raw-program cache, described next.

8.2 Persisting the FORM expansion before projection

index/form_expansion_cache.py now owns IndexFormTerm, the exact parser, and FormExpansionCache. For raw program text P, the stored value is the parsed list of monomials before any gauge projection:

$$P \longmapsto \{(c, \alpha, \beta, \gamma, ((p, \mathbf{n}_p))_p)\}\,, \qquad c\, t^{\alpha} y^{\beta} u^{\gamma} \prod_{p,j} C_p(j)^{n_{p,j}}.$$

This notation is a direct description of the project record format. Each IndexFormTerm has a Fraction coefficient, integer fugacity powers, and a tuple of character indices and Adams-power tuples. The compact JSON row is ["numerator","denominator",t_power,y_power,u_power,characters]. _encode_expansion stores a list of these rows; _decode_expansion restores exact arbitrary-size fractions and nested tuples. No float conversion or representation decomposition occurs in this layer.

```
CREATE TABLE form_expansions (
    program TEXT COLLATE BINARY NOT NULL PRIMARY KEY,
    expansion_json TEXT NOT NULL
) WITHOUT ROWID;
```

The key is the complete program text, compared verbatim, not a hash. Whitespace, numeric multiplicities, symbol names and the truncation order matter. There is no program normalization or symbolic-multiplicity template. The default file is form_expansion_cache.db, separate from the character database; the FORM module does not set a separate user_version.

Lookup. get_expansion(program) reads and decodes a hit. A miss runs run_form, calls parse_form_output, encodes the terms and commits with ONCONFLICT(program)DONOTHING. Failed execution or parsing does not create an entry. Each thread opens its own connection, with a PID check before reusing it, WAL, a 30-second busy timeout and up to three attempts for busy/locked writes. FORM runs outside write transactions. Concurrent identical misses can execute FORM more than once, but retain only one complete row. The context manager closes the current thread's connection. These boundaries follow the implementation and SQLite's serialized-writer model [8, 10].

Index integration. calculate_index and calculate_index_internal accept form_cache_database_path=; the index CLI exposes --form-cache-database. Otherwise the FORM database is placed beside the selected character database. A hit bypasses FORM execution and FORM-output parsing, while group-specific singlet projection still runs. Orders below two return the vacuum without external execution or cache access.

Different theories can share this expansion when their raw programs are identical and they use the same FORM database. Group and irrep meanings are supplied afterward by each theory's character basis. Their final indices can therefore differ despite a shared expansion. This reuse applies to the supported non-SU groups too; it does not require a power-sum/Schur integration backend.

8.3 Reusing lower-order character decompositions

Write the cached virtual character as

$$D_R(\mathbf{n}) = \sum_{\lambda} d_\lambda \chi_\lambda = \prod_{j\geq 1} (\psi^j \chi_R)^{n_j}, \qquad q(\mathbf{n}) = \sum_j j n_j.$$

If $\mathbf{a} + \mathbf{b} = \mathbf{n}$ componentwise, multiplication in the representation ring gives

$$D_R(\mathbf{n}) = D_R(\mathbf{a}) D_R(\mathbf{b}), \qquad d_\nu = \sum_{\lambda,\mu} a_\lambda b_\mu N^\nu_{\lambda\mu}, \quad \chi_\lambda \chi_\mu = \sum_\nu N^\nu_{\lambda\mu} \chi_\nu.$$

The identity follows by multiplying the defining Adams products; LiE's `tensor` implements the last decomposition [3, Sec. 4.5]. Signed coefficients remain valid. The two exact base cases are $D_R(\mathbf{e}_1) = \chi_R$ and $D_{\mathbf{1}}(\mathbf{n}) = 1$; neither needs a LiE call.

`CharacterDecompositionCache._decomposition_dependencies` first tries the usual split: remove one factor at the smallest occupied Adams index. For at most two factors, or when both usual dependencies are available, it retains that split. Otherwise it searches lower-weighted-order keys for the *same* Cartan type and Dynkin labels, including thread-local entries and the known first Adams operation. It considers pairs already available whose padded exponent vectors sum to the requested vector.

Among alternative pairs, the score is the product of the two numbers of irreducible terms, with zero cost for a zero or scalar-trivial factor. Ties use the total number of terms and then the vectors themselves. This estimates tensor work; it does not predict the fastest LiE call or find a globally optimal factorization. The batch dependency planner and the actual calculation use the same selection. Existing SQLite keys, schema and thread/process ownership are unchanged.

For example, if $R^{\otimes 2}$ and $(\psi^2 R)^{\otimes 2}$ are stored, their product can be obtained with one tensor operation on those entries. Atomic prerequisites need not be recalculated just to reconstruct a cached composite. On-demand generation still constructs missing prerequisites when no suitable cached split is available. It does not automatically enumerate all lower orders.

Scope. This optimization is separate from the singlet splitter. The singlet path still decomposes two partial products and contracts dual labels; the new dependency choice helps construct the required same-irrep partial products. Intermediate full tensor decompositions can remain expensive even when all individual Adams operations have already been cached.

8.4 Building every product through a chosen Adams order

`build_decomposition_cache(cartan_type,dynkin_labels,max_adams_order,...)` in `index/char_decomposition_cache.py` fills the existing decomposition table for one base irrep. At weighted order q it calls `common.math_utils.frobenius_solve(range(1,q+1),q)` to enumerate

$$\mathcal{P}_q = \left\{ (n_1, \ldots, n_q) \in \mathbb{Z}^q_{\geq 0} : \sum_{j=1}^{q} j n_j = q \right\}.$$

Each solution specifies one Adams product. This is also the multiplicity description of an integer partition: $|\mathcal{P}_q| = p(q)$, with counts $1, 2, 3, 5, 7, 11$ at orders one through six (29 rows in total).

Orders run sequentially. The builder reads existing keys at the current order, skips complete rows without decoding their payloads, and divides missing solutions into batches for a persistent `ProcessPoolExecutor` using `spawn`. Batches contain at most 64 requests, with several batches per worker to balance uneven costs. Workers read prerequisites and calculate; the parent commits each returned batch. It submits the next order only after the current one is fully committed. LiE never runs in a write transaction. Completed batches survive a later failure or interruption, so a rerun resumes from the missing entries. Independent builders may duplicate simultaneous misses; uniqueness and short transactions preserve complete stored rows.

Every composite at order q has a split into two strictly lower orders. Consequently both factors are already stored and at most one tensor operation is needed for that request, with scalar/zero shortcuts as usual. For an initially empty cache of a nontrivial irrep, one uninterrupted build needs only one individual Adams calculation per $j = 2, \ldots, M$: $M - 1$ in total. This counts logical Adams calculations, not backend retries or concurrent duplicate builds. The builder fills same-irrep decompositions, not singlet coefficients, mixed-irrep products or the FORM database.

Cutoff for the full index. A letter starts at t^2, and its j-th Adams contribution starts no earlier than t^{2j}. For each base irrep in a fixed gauge factor, its product therefore satisfies

$$\deg_t \;\geq\; 2\sum_j j n_j = 2q \quad\Longrightarrow\quad q \leq \left\lfloor \frac{N}{2} \right\rfloor \quad \text{for an index through } t^N.$$

This is a bound derived from the implemented single-letter functions. Use $M = \lfloor N/2 \rfloor$ for each required adjoint and matter irrep, retaining distinct conjugate labels. It is a sufficient bound, not a claim that every partition will occur in the projector. For product groups it holds factorwise; one bifundamental letter contributes a character to multiple factors, so one must not sum their separate orders and apply the same bound to that sum. Through t^{18}, $M = 9$ suffices. For $N < 2$ there is no cache to prebuild.

8.5 Builder API, dependencies and command-line examples

With Sage's environment active, prepare the A_2 fundamental through order nine:

```
sage -python -m index.char_decomposition_cache A2 \
  --dynkin-labels 1 0 --max-adams-order 9 --processes 4 \
  --cache-database /tmp/index-demo/char_decomposition_cache.db
```

Repeat for every required irrep. For A_2 SQCD these include $(0,1)$ and the adjoint $(1,1)$ in addition to $(1,0)$. The group rank does not multiply the maximum Adams order. The direct script entry point `sage -pythonindex/char_decomposition_cache.py...` is also supported.

```
from index.char_decomposition_cache import build_decomposition_cache

if __name__ == "__main__":
    totals = build_decomposition_cache(
        "A2", (1, 0), 9, processes=4,
        database_path="/tmp/index-demo/char_decomposition_cache.db",
        progress=lambda q, total, computed:
            print(q, total, computed),
    )
```

The return value is `dict[int,int]` mapping each order to its total product count, including reused rows. The optional callback receives `(order,total,computed)` in the parent after that order is committed. `processes=1` is serial; the default is the available CPU count. The pool starts only when there is parallel work. Use a `__main__` guard in scripts with multiple processes.

The maximum order and worker count must be positive integers. Cartan type, rank and Dynkin labels are validated. `cache_directory=` and `database_path=` are mutually exclusive. Other keywords are `lie_executable=` and `timeout=` (600 seconds per LiE call by default; Python accepts `None` for no timeout). The CLI provides `--max-order` as an alias for `--max-adams-order`, as well as `--cache-directory`, `--lie-executable`, and numeric `--timeout`. It prints computed/reused counts and exits with status 2 on a reported failure.

The builder imports `frobenius_solve` lazily and therefore needs OR-Tools only when called; ordinary index/cache lookups do not acquire that dependency. Its enumeration and complete decompositions can grow much faster than the subset used by one index. Prebuilding is optional, and a fully warm repeat still enumerates solutions and checks keys even though it runs no LiE or worker pool.

Use the prepared character database and a shared FORM database for index calls:

```
sage -python -m index.n2_theory_index theory.json --order 18 \
  --cache-database /tmp/index-demo/char_decomposition_cache.db \
  --form-cache-database /tmp/index-demo/form_expansion_cache.db
```

Without the last option, the FORM database defaults to that same directory. The corresponding index Python keyword is `form_cache_database_path=`.

9 Python functions and their mathematical roles

Function or class	Mathematical/computational role
`_parse_input`	Resolves the ordered factors in Eq. (1) and validated hypermultiplet data.
`_matter_character_multiplicities`	Constructs Eq. (10), omits trivial factor characters, and applies conjugation as in Eq. (11).
`_character_basis`	Assigns each pair (a, R) one FORM function C_p and records vector and matter character monomials.
`_build_form_program`	Implements Eqs. (7), (17), (14), and (20).
`run_form`	Shared helper in `common/form_utils.py`; runs FORM in an isolated temporary directory and requires a successful, clean process result.
`FormExpansionCache.get_expansion`	Retrieves parsed terms by raw program; executes, parses and stores a miss.
`FormExpansionCache.parse_form_output`	Converts exact FORM monomials to `IndexFormTerm` records with `Fraction` coefficients.
`get_decomposition`, `get_decompositions`	Retrieve or construct Eq. (21); prefer available cached factors and batch missing prerequisites by tensor-factor count.
`build_decomposition_cache`	Enumerate all products at each weighted order; commit lower orders before parallel generation of the next.
`get_singlet_multiplicities`	Look up symbolic-product singlets first; calculate missing integers with `_character_singlets`.
`singlet_multiplicities`	Cache singlets from already-decomposed inputs; misses call `_compute_singlet_multiplicities`.
`_split_character_product`	Greedily split individual Adams factors using Eq. (28c). Repeated factors may separate.
`_character_singlets`	Internal `plan()` identifies needed parts; `expand()` builds and memoizes mixed parts before final pairing.
`_dual_permutation`, `_singlet_pair`	Cache the label permutation in Eq. (28d) and evaluate Eq. (28b), including signed coefficients.
`_tensor_decompositions`	LiE decomposition of intermediate products; zero and scalar-trivial factors are handled directly.
`_compute_singlet_multiplicities`	For decomposed inputs, sort by support size, recursively build two parts and pair dual labels.
`_project_terms`	Multiplies factorwise singlet multiplicities and collects equal powers of t, y, u.
`_to_sage_polynomial`	Returns an exact flat Laurent polynomial in $\mathbb{Q}[t^{\pm1}, y^{\pm1}, u^{\pm1}]$; performs no representation calculation.
`calculate_index`	Coordinates the complete public in-memory calculation.
`calculate_index_from_file`	Loads JSON and delegates to `calculate_index`.

10 JSON examples and command-line use

A simple-factor theory retains the existing form:

```
{
    "algebra": "A2",
    "hypermultiplets": [
      {"representation": "fundamental", "number": 6, "kind": "full"}
    ]
}
```

A product theory names every gauge factor. Missing entries in `representations` mean singlets:

```
{
    "gauge_groups": [
       {"id": "left", "algebra": "A1"},
       {"id": "right", "algebra": "A1"}
    ],
    "hypermultiplets": [{
       "representations": {
          "left": "fundamental", "right": "fundamental"
       },
       "number": 2, "kind": "full"
    }]
}
```

With the Sage environment active and `form` and `lie` on `PATH`:

```
python -m index.n2_theory_index theory.json --order 10
```

The default files are `char_decomposition_cache.db` and `form_expansion_cache.db` at the project root. Use `--cache-database /path/to/cache.db` for an explicit file, or `--cache-directory /path/to/directory` for the default filename inside that directory. These options are mutually exclusive; the corresponding Python arguments are `database_path=` and `cache_directory=`. The FORM database follows the character database directory unless `--form-cache-database` (or Python `form_cache_database_path=`) overrides it.

For example, an unlimited single-worker calculation through t^{18} is

```
result = calculate_index(
    data, 18, processes=1, timeout=None,
    database_path="/tmp/fresh-index/characters.db",
    form_cache_database_path="/tmp/fresh-index/form.db",
)
```

Use previously nonexistent files for both databases for a fully cold-cache timing. Filling only the character database leaves FORM cold, and vice versa. `timeout=None` is passed through to FORM and LiE; it removes their subprocess timeout.

11 Validation and representative performance

11.1 Earlier backend replacement checks (1 September snapshot)

The former sparse Sage weight-lattice implementation was retained long enough to serve as an independent reference before replacement. Exact comparisons gave:

Theory	Order	Check
$SU(2)$ with four full fundamentals	t^{18}	exact agreement
$SU(3)$ with six full fundamentals	t^{18}	exact agreement
$SU(4)$ with eight full fundamentals	t^{18}	exact agreement
$SU(2) \times SU(2)$ bifundamental theory	t^{10}	independent exact agreement
Decoupled product theories	t^6	factorization identity
$SU(2)^3$ trifundamental half hyper	t^6	factor-order invariance

For conformal $SU(3)$ SQCD, the first nonzero terms are

$$\begin{aligned}\mathcal{I} = 1 + t^4\left(\frac{36}{u^2} + u^4\right) - t^5u^2(y + y^{-1}) \\ + t^6\left(\frac{40}{u^3} - 36 + u^6\right) + O(t^7).\end{aligned} \tag{30}$$

Representative product-group timings through t^{10} were:

Theory	Cold cache	Warm cache
$A_1 \times A_1$ bifundamental	1.030 s	0.042 s
$A_1 \times G_2$ mixed matter	0.274 s	0.127 s

These historical timings predate FORM caching: their warm runs remove repeated LiE work but still execute the FORM series expansion. They are machine-dependent. The complete project test run after that earlier replacement passed 55 tests; two live MySQL integration tests were skipped because no test database was requested.

11.2 SQLite and sparse-projection checks (9 September)

The temporary projection candidate matched all seven comparative index cases and 319 singlet queries against the previous SQLite implementation (`a9b7009`). Of these, 202 also matched independent $SU(2)/SU(3)$ weight calculations. All 216 dual-label comparisons across 18 Cartan types matched Sage. All 4,996 old SQLite singlet entries tested were reused without recalculation. These checks preceded promotion to `0a402ec`, before the later FORM/planner/builder changes.

The full post-projection suite ran 170 tests in 11.461 s: 168 passed and two live MySQL tests were skipped. Seven new regressions cover signed complex/pseudoreal pairing, grouped Adams splitting, mixed products, independent weights, parallel generation, trivial factors and zeros. These are local validation results; they do not establish correctness at all ranks and cutoffs.

11.3 Earlier speedups and A32 profile, before FORM caching

Cold calculation	Previous (s)	Revised (s)
A_8, t^{18}, singlet projection	4.599	0.849
A_{16}, t^{12}, singlet projection	1.473	0.207
A_{32}, t^8, singlet projection	0.469	0.117
A_8, t^{18}, complete index	6.213	2.443

These are three-trial medians with fresh cache files and one worker, for SQCD with $2(r+1)$ full fundamental hypers. Projection excludes the shared FORM expansion. Warm A8 complete-index time remained about 1.70 s. Four-worker comparisons also matched, though process startup can slow small cases.

On 10 September, $A_{32} = SU(33)$ with 66 full fundamental hypers completed through t^{18} with no timeout and one worker:

Stage	Cold (s)	Warm (s)
Input/validation/algebra initialization	4.483	0.009
FORM expansion	1.357	1.406
FORM output parsing	0.212	0.239
Singlet projection, including cache work	88.229	0.127
Complete calculation	94.293	1.786

One LiE call decomposing $\psi^3(\text{adj})^2$ took 82.699 s, or 87.7% of the cold total. Its inputs each have 10 irrep terms and its output has 91. All other LiE calls together took 5.066 s. The warm run made no LiE calls. Both index results contain 117 nonzero monomials and match exactly. This is one wall-clock profile, not a median; the warm run shares Sage initialization but uses a new cache client.

Evidence and reproduction commands are in `output/benchmarks/singlet_projection_benchmark.md` and `output/benchmarks/a32_unlimited/report.md`. The unlimited run shows that the remaining cost is an intermediate full decomposition, not the final sparse pairing.

11.4 FORM cache measurements with character projection prefilled

The controlled comparison used nine theories at t^{12} and t^{18}, one warmup and five measured repetitions per mode, with fresh clients and one worker. Both decompositions and final singlet coefficients were already filled. The character database was query-only and guards prohibited LiE execution. All 270 timed indices agreed exactly. Medians below are full calculation times in seconds, excluding setup and Python/Sage startup.

Theory, through t^{18}	Disabled	Miss/save	Hit	Speedup
$SU(2)$, 4 fundamentals	0.4323	0.4916	0.0617	7.00×
$SU(3)$, 6 fundamentals	1.7220	1.7959	0.2304	7.48×
$SU(5)$, 10 fundamentals	1.7265	1.7839	0.2648	6.52×
$SU(3)$, 1 adjoint ($\mathcal{N} = 4$)	0.2808	0.3180	0.0429	6.54×
$Sp(2)$, 6 fundamentals	0.4434	0.4817	0.0634	7.00×
Spin(8), 6 vectors	0.4329	0.4848	0.0655	6.60×
G_2, 4 fundamentals	0.4418	0.4874	0.0640	6.90×
$SU(2)^2$, 2 bifundamentals	2.7421	2.7843	0.3251	8.44×
$SU(5)$, symmetric + antisymmetric	13.7565	14.1896	1.8771	7.33×

All matter in this table is full hypermultiplets; $Sp(2) = USp(4)$. Disabled mode bypasses FORM SQLite and executes/parses FORM; miss/save begins with an initialized empty FORM database and includes serialization and insertion. Hit mode decodes an existing expansion with FORM execution forbidden. The speedup is disabled time divided by hit time. At t^{12} the range was 3.23–6.92×. These are warm-filesystem measurements on this machine, not cold-disk or contention benchmarks. Evidence: `report.md`, `results.json` and `benchmark.py` under `output/benchmarks/form_expansion_theories/`.

Cross-theory hits. Two pairs share byte-identical programs: $SU(2)$ with four fundamentals and G_2 with four fundamentals; $Sp(2)$ with six fundamentals and Spin(8) with six vectors. Both directions at both orders gave eight passing reuse checks. The first theory executed FORM once; a new client for the second executed/parsed FORM zero times, decoded once, and left the database at one row. Each final index matched its own independent fresh-FORM result, while the paired final indices differed. A changed-program control missed as expected. See `cross_theory_reuse.md`, `cross_theory_reuse.json` and `check_cross_theory_reuse.py` in that same directory.

11.5 I/O cost, decomposition planning and builder verification

A separate I/O benchmark used the 113,831-term t^{18} expansion: a 589-byte program and 5,907,568-byte JSON payload (5.91 MB). Thirty measured repetitions followed three warmups, using SQLite WAL with synchronous FULL on the project filesystem. Medians are milliseconds:

Operation	Median (ms)
Serialize exact terms	179.32
SQLite insert and commit	42.07
Total save, excluding database initialization	221.51
SQLite read and fetch JSON	3.08
Reconstruct `IndexFormTerm` objects	866.42
Hit with an already open connection	870.81
Hit with a new client, including close	894.65

Initialization separately took 31.43 ms. No FORM execution/parsing is included; every round trip was checked exactly. Python reconstruction dominated loading. Individual component medians need not add to the median total. This is a single-process warm-filesystem test, not concurrent lock contention. Evidence: `output/benchmarks/form_expansion_cache_io/report.md`, `timings.json` and `benchmark.py`.

Cached-factor planner. The temporary candidate matched 130 products over A_1, A_2 fundamental/adjoint, C_2 and G_2 in serial and three-worker runs; $SU(2)$ cases also matched an independent weight oracle. Prepared-cache misses improved 1.31–1.72× for the single-request API and 1.03–1.17× for the batch API, reducing two tensor calls to one. A case with only composite entries also reduced one Adams call to zero. However, 72 full-index comparisons over six theories, with FORM prefilled and either a cold character cache or a cache filled through t^{12}, showed *no consistent overall speedup*. All indices agreed and tensor-call counts were unchanged. The retained baseline, candidate, report, scripts and raw measurements are in `output/benchmarks/adams_cache_planner/`.

Complete builder. Twelve regressions cover all 29 $SU(2)$ products through order six against independent weights, serial/parallel agreement for A_1, A_2, C_2, G_2 through order five, pool reuse, warm skips, partial-cache resume, failure persistence, input validation and both CLI entry points. A real worker log records exactly one Adams call at each of orders two through six and 23 tensor calls, with multiple worker PIDs. This verifies the operation-count claim on the tested build; no broad prebuild speedup has been measured.

The full suite rerun during the 10 September 2026 cache-documentation update ran **209 tests: 207 passed and two live-MySQL tests skipped**. It uses actual Sage/FORM/LiE for backend tests; MySQL migration and insertion unit tests use mocks unless a test server is explicitly configured. The new modules are `test_form_expansion_cache.py`, `test_cached_decomposition_planning.py` and `test_build_decomposition_cache.py`. These measurements and algorithms are source-derived project evidence, not results attributed to external papers.

12 Property calculation and staged database updates

The direct `index.n2_theory_index.calculate_index(data,order,...)` API still computes a formal gauge projection; it is distinct from the SCFT-gated property layer and the database importer. Their workflow is now split:

- `calculate_n2_theory_properties(data)` returns group/candidate flags, flavor symmetry, marginal gauge couplings, conformal-manifold dimension and exact central charges. It omits both indices and the Coulomb spectrum.
- `calculate_n2_theory_indices(data,order=18,max_dimension=90)` separately returns the full index, Coulomb index and complete spectrum, plus `superconformal_index_order` and `coulomb_branch_index_max_dimension`. Both index values are strings; the spectrum contains exact rational dimensions with repetitions.

Malformed input raises an error. For valid non-SCFT candidates the basic layer retains flavor metadata, but the separate index layer returns None for all five index-related fields. The basic CLI is unchanged in name; `--indices` selects the new calculation stage, with optional `--index-order` and `--coulomb-max-dimension`.

The Coulomb index is computed directly from the gauge-invariant degrees, independently of the full-index cutoff. In the project convention the Coulomb ray is $t^{2\Delta}u^{2\Delta} = x^{\Delta}$; a full index known through t^{18} can therefore justify Coulomb terms only through dimension nine. The separate calculation through dimension 90 does not extract extra information from that truncated full index.

12.1 Independent precision and replacement rules

The database stores one set of physical properties per theory. A Lagrangian realization supplies the input for later calculation; different realizations attached to the same theory share its indices and Coulomb spectrum. Initial imports run the anomaly/conformality checks and calculate basic properties only. Both indices, the spectrum and both precision columns start as SQL `NULL`; their keys are initially absent from `properties_json`.

The separate calculator returns full-index precision $N \in \mathbb{Z}_{\geq 0}$ and Coulomb precision $D \in \mathbb{Q}_{\geq 0}$. These are inclusive cutoffs:

$$\mathcal{I}_{\leq N} = \sum_{a=0}^{N} t^a P_a(y,u), \qquad \mathcal{I}_{\mathrm{C},\leq D} = \sum_{0\leq\Delta\leq D} c_\Delta x^\Delta.$$

A zero boundary coefficient does not reduce the trusted cutoff. For example, $1 + x^2 + x^4$ computed through $D = 5$ must retain precision five, even though its largest nonzero exponent is four. The complete gauge-invariant generator spectrum is independent of either cutoff; it is not truncated with the index.

For each of the two indices separately, let V be its stored value and $p_{\mathrm{old}}, p_{\mathrm{new}}$ its stored and submitted cutoffs. The write rule is

$$\text{replace or fill} \iff V \text{ is missing} \quad \text{or} \quad \big(p_{\mathrm{old}} \text{ is known and } p_{\mathrm{new}} > p_{\mathrm{old}}\big).$$

Stored / submitted state	Database action
Stored index missing	Fill it and record the supplied cutoff.
Known, strictly higher cutoff	Replace that index and its cutoff.
Equal or lower cutoff	Keep the existing string and precision unchanged.
Legacy index, unknown cutoff	Keep it; report `unknown_precision`.
Omitted or `None` component	Leave the corresponding stored data alone.
Supplied index without a cutoff	Reject the payload before opening the write transaction.

For example, an update from $(N, D) = (18, 90)$ to $(24, 60)$ upgrades the full index only. The Coulomb index remains at dimension 90. Equal-order submissions do not replace even a different string; this API is not a correction/force-update mechanism. The caller supplies the trusted calculation cutoff; the writer does not infer or prove precision from the expression's last monomial.

A missing spectrum is filled. An identical complete spectrum is retained, with multiplicities preserved; a conflicting spectrum raises an error and rolls back that update call. This policy differs from comparison of truncated indices.

Implementation source: `common/n2_theory_properties.py`, `common/n2_theory_db.py`; the inequalities above express the implemented cutoff contract, not an additional physical conjecture.

12.2 Schema 7, legacy data and atomic writes

Schema 7 keeps the same ten tables and adds two nullable columns to `theory_properties`. Central charges and all existing index data are preserved by the migration.

Column / result key	Stored representation
`superconformal_index_order`	`BIGINT UNSIGNED`; inclusive integer N.
`coulomb_branch_index_max_dimension_json`	Exact numerator/denominator JSON for D.
`coulomb_branch_index_max_dimension`	Corresponding key in calculator output and combined JSON.

The three computed values still use `superconformal_index_json`, `coulomb_branch_index_json` and `coulomb_branch_spectrum_json`. Successful updates mirror changed values and cutoffs into `properties_json` while preserving basic properties and other computed components.

Migration 6 → 7 adds the columns without guessing legacy cutoffs, recomputing indices, rehashing realizations or merging theories. Initialization applies migrations when the database is next opened through the database API. MySQL schema DDL precedes the explicit data transaction and is not rolled back with it.

When an old result's original requested cutoff is independently known, record it before normal upgrades:

```
from common.n2_theory_db import record_lagrangian_index_cutoffs

# Use these numbers only if they are the verified original cutoffs.
record_lagrangian_index_cutoffs(
    connection, realization_id, order=18, max_dimension=90,
)
```

This function fills unknown precision only. It rejects changing an already known cutoff or annotating an index that is absent. It changes neither index expression. Merely seeing a last term of degree 18 does not establish that the original calculation was requested through 18.

`update_lagrangian_indices(connection,realization_id,indices)` accepts `calculate_n2_theory_indices`'s output or a subset of computed components with their matching cutoffs. It opens a short transaction, locks the shared row with `SELECT ... FOR UPDATE`, rechecks the current cutoffs, and merges accepted changes into the columns and JSON together. Failure rolls back the call; a no-op does not change the theory's `updated_at` timestamp.

Calculations run before this lock is acquired. Thus a slower low-order worker cannot overwrite a higher-order result committed while it was calculating. Use an initialized connection with no caller-owned transaction for update and legacy-cutoff APIs. These APIs manage their own commit/rollback boundary.

12.3 Separate worker, partial retries and reproducible use

The program common/n2_theory_db_indices.py streams stored Lagrangian inputs in bounded pages, using the smallest realization ID for each theory. It visits each shared theory once rather than recalculating the same shared properties for every attached realization.

1. Default mode selects any missing full index, Coulomb index or spectrum. SQL NULL and JSON null are both treated as missing.
2. --upgrade additionally selects indices with known lower cutoffs. Already sufficient components are not calculated. Complete legacy records with unknown precision are reported without recalculating those indices.
3. Each requested component is calculated outside a database transaction, then submitted to the locked update API. The stored state is checked again at write time, even if it differed when the job was selected.
4. Each successful component commits independently. A failure is reported for that component; subsequent components and theories continue. Re-running retries remaining missing or lower-order components and keeps earlier successes.

Multiple workers can duplicate a calculation; there is no persistent job-claim queue. The row-lock and cutoff policy protects data against downgrades, not against duplicated CPU work. A conflicting spectrum rolls back its own update call, not components already saved by earlier worker calls.

Run from the repository root with Sage's Python. Database credentials follow the existing N2_DB_HOST, N2_DB_USER, N2_DB_PASSWORD and N2_DB_UNIX_SOCKET environment conventions; port is a CLI option.

```
# Stage 1: validate and store basic properties.
sage -python -m common.n2_theory_db \
  anomalies/example_e6.json DATABASE --user USER
# Stage 2: fill missing components, defaults N=18 and D=90.
sage -python -m common.n2_theory_db_indices DATABASE --user USER
# Later: upgrade only components with lower known precision.
sage -python -m common.n2_theory_db_indices DATABASE --user USER \
  --upgrade --index-order 24 --coulomb-max-dimension 100 --limit 10
```

The worker also accepts --host, --port, --unix-socket, --connect-timeout, --cache-directory, --lie-executable, --form-executable, --timeout and --processes. --limit bounds visited theory jobs, including reported legacy skips. The API fill_lagrangian_indices yields the same per-theory outcomes. Each outcome reports updated_fields, skipped_fields and errors; a final CLI summary counts processed and failed jobs. Exit codes are 0 for no component failures, 1 for component failures, and 2 for an outer configuration or database-level error.

Validation recorded during implementation, 12 September 2026. All 238 backend tests passed in 21.920 seconds, including an isolated MySQL 8.0.46 server, schema migration, concurrent upgrades, rollback, partial retries, JSON-null selection and real Sage/FORM/LiE calculations. This PDF update checks source contracts and document rendering; it does not rerun that suite or alter the user's database. Older cache-performance measurements remain dated evidence.

13 Interpretation and limitations

- The result is a signed protected index, not a complete operator spectrum.
- Flavor fugacities are unrefined. Matter multiplicities are integer coefficients.
- The code supports semisimple products of the listed finite simple algebras, but not $U(1)$ gauge factors.
- The Lie algebra does not determine the global form of the gauge group, discrete theta data, or the spectrum of line operators.
- Input validation checks representation syntax and physical admissibility of a half hyper. The index routine does not require the beta functions or global gauge anomalies to vanish; those are separate checker outputs.
- High orders still produce many FORM monomials and expensive intermediate LiE products. The cache reduces repeated work, and sparse pairing avoids the complete product's decomposition; neither removes all combinatorial growth or guarantees an optimal intermediate split.

References

[1] A. Gadde, E. Pomoni, and L. Rastelli, "The Veneziano Limit of $\mathcal{N} = 2$ Superconformal QCD: Towards the String Dual of $\mathcal{N} = 2$ $SU(N_c)$ SYM with $N_f = 2N_c$," JHEP 03 (2010) 032, arXiv:0912.4918.

[2] S. Benvenuti, B. Feng, A. Hanany, and Y.-H. He, "Counting BPS Operators in Gauge Theories: Quivers, Syzygies and Plethystics," JHEP 11 (2007) 050, arXiv:hep-th/0608050.

[3] M. A. A. van Leeuwen, A. M. Cohen, and B. Lisser, *LiE, A Package for Lie Group Computations*, manual, 1992. In particular, Secs. 3.4 and 4.5 describe Adams operations and tensor-product decomposition polynomials. CWI manual record.

[4] J. Kuipers, J. A. M. Vermaseren, and collaborators, *FORM 5.0.0 Reference Manual*, 2026.

[5] J. Kuipers, T. Ueda, J. A. M. Vermaseren, and J. Vollinga, "FORM version 4.0," Comput. Phys. Commun. 184 (2013) 1453–1467, arXiv:1203.6543.

[6] P. Etingof, *Lie Groups and Lie Algebras*, MIT 18.755 lecture notes (2024), Proposition 32.1 and Corollary 35.9. Official MIT notes.

[7] The Sage Developers, *Weyl character rings*, `dual()` and `inner_product()`. Official Sage documentation.

[8] SQLite, *Write-Ahead Logging*, §2.2. Official WAL documentation.

[9] SQLite, *Datatypes In SQLite*, §2. Official datatype documentation.

[10] Python Software Foundation, *sqlite3: DB-API 2.0 interface for SQLite databases*. Official Python documentation.

New references were checked on 10 September 2026. The splitting heuristic, function map and measurements are derived from the project source and local validation, rather than attributed to the representation-theory literature.